



GTVD: a multi-level aggregation vulnerability detection method based on full-dependency program graph

Haitao He¹ · Shibo Li¹ · Yuxiang Li¹ · Yang Li¹

Received: 10 November 2024 / Revised: 6 May 2025 / Accepted: 7 May 2025 / Published online: 3 September 2025
© The Author(s), under exclusive licence to Springer Science+Business Media, LLC, part of Springer Nature 2025

Abstract

In modern software development life cycles, proactive vulnerability discovery and remediation play crucial roles in ensuring application security. However, current deep learning-based vulnerability detection methods frequently face limitations due to overly simplistic feature extraction procedures and inadequate handling of long-range dependency relationships. In this paper, we present GTVD, a graph-based vulnerability detection framework for C/C++ source code, which addresses these challenges through three key innovations. First, we introduce the Full Dependency Program Graph (FDPG), a novel intermediate representation that comprehensively encodes both syntactic structures and semantic relationships within source code. This advancement overcomes the feature representation constraints inherent in conventional code attribute graphs. Our architecture employs a hierarchical Graph Neural Network to systematically extract structural patterns from the FDPG representation, ensuring a robust analysis of the program's inherent structures. At the core of our feature extraction mechanism lies the Multi-Level Message Aggregation (MLMA) strategy. This innovative approach enables progressive integration of information across multiple neighborhood orders, effectively capturing both local and global program dependencies. To mitigate feature degradation in long-range dependencies, we develop an Adaptive Weighted Aggregation (WAG) mechanism that dynamically adjusts feature contributions during graph-level representation learning. Comprehensive evaluations on three large-scale public datasets demonstrate GTVD's superior performance. Our method achieves an average improvement of 7.76% across four evaluation metrics compared to the baseline, thereby confirming our method's enhanced capability to identify complex vulnerability patterns.

Keywords Vulnerability detection · Full dependency program graph · Multi-level message aggregation · Adaptive weighted aggregation

1 Introduction

Software systems are integral to critical sectors such as finance, healthcare, and national defense. As reliance on these systems intensifies, the need for effective software

vulnerability detection and prevention becomes increasingly critical. Data from the National Information Security Vulnerability Database (CNNVD) reveal that in April 2024 alone, 3665 new security vulnerabilities were reported—a notable increase from the monthly average of 2902. Furthermore, the incidence of super-critical vulnerabilities, which pose severe risks to both the economy and society, is on the rise.

As the number of software products continues to grow, so does the prevalence of software vulnerabilities, which are expected to increase in the near future [1]. These vulnerabilities may occur in various types of software, including operating systems, applications, database management systems, and network services, thereby posing serious security risks. Developers and security engineers primarily employ code analysis and testing tools—such as

✉ Shibo Li
lvbtale@stumail.ysu.edu.cn

Haitao He
haitao@ysu.edu.cn

Yuxiang Li
2467526488@stumail.ysu.edu.cn

Yang Li
chan@stumail.ysu.edu.cn

¹ Yanshan University, Qinhuangdao, China

rule-based analysis and symbolic execution [2]—to detect and remedy errors. However, because these methods are prone to false positives, they often require substantial manual inspection for verification. Although manual auditing methods offer high flexibility, they are time-consuming, costly, and depend on the involvement of senior experts, which can easily result in overlooked vulnerabilities.

With the rapid development of artificial intelligence and machine learning technologies, software vulnerability detection methods leveraging these approaches are gradually emerging. By utilizing machine learning and deep learning algorithms, new vulnerability patterns can be automatically identified, thereby enhancing the intelligence and automation levels of detection. As a result, software vulnerability detection in these emerging fields has become a prominent research topic. Studies [3] have reviewed various methods for extracting source code features and classification techniques, noting that binary code representations often lack grammatical and semantic information, which hampers detection accuracy. In contrast, dynamic analysis monitors program behavior in real time, capturing issues that arise under specific conditions and offering a more accurate reflection of program vulnerabilities. Nonetheless, dynamic analysis faces challenges such as path explosion. Furthermore, research [4, 5] has applied natural language processing techniques to transform code into an intermediate representation, aiming to refine feature extraction and binary classification of vulnerabilities [6]—a crucial but challenging step due to feature loss over long distances.

To address the limitations of code representation diversity and feature information loss, we introduced the Full Dependency Program Graph (FDPG), which provides a comprehensive representation of source code by integrating code attribute graphs. During the feature extraction stage, we employed graph neural networks (GNNs) and developed the Multi-Level Message Aggregation (MLMA) module to effectively extract detailed feature information from each node. Additionally, the Adaptive Weighted Aggregation (WAG) module was designed to dynamically weight node-level features through an attention mechanism, thereby enhancing graph-level feature integration. Finally, these graph-level features are fed into a fully connected layer to perform vulnerability detection tasks.

The contributions of this paper are as follows:

- A novel intermediate source code representation, named the Full Dependency Program Graph (FDPG), is proposed. This method comprehensively incorporates lexical, semantic, and syntactic correlations while preserving the grammatical features of source code,

thereby addressing the limitations of insufficient feature representation in existing approaches.

- A Multi-Level Message Aggregation (MLMA) method is introduced to mitigate information attenuation in traditional graph neural networks during feature propagation. This approach enables nodes to access and integrate data beyond their immediate neighbors within a single iteration, effectively expanding each node's perceptual range and ensuring a deeper, broader information flow throughout the graph.
- An Adaptive Weighted Aggregation (WAG) strategy is designed to allow the network to dynamically adjust node weights based on their contextual contributions to the global graph structure. This mechanism significantly enhances the model's capability to capture intricate graph patterns.

The remainder of this paper is structured as follows: Sect. 2 reviews existing deep learning approaches to vulnerability detection. Section 3 details the proposed GTVD model and its components, designed for detecting vulnerabilities in C/C++ source code. Section 4 outlines the experimental setup and presents the evaluation results. Section 5 concludes the paper and discusses potential directions for future research.

2 Related work

In this section, we discuss the principal methods employed in vulnerability detection, encompassing three main approaches: sequence-based, graph-based, and large language model-based methodologies.

2.1 Sequence-based methods

Due to its widespread adoption, deep learning has been extensively applied to vulnerability detection. Initially, the approach involved processing code as token sequences for analysis; however, a critical issue emerged: models were unable to comprehend code semantics in the manner that humans do. Consequently, an intermediate representation that retains essential grammatical and semantic information is necessary to capture these features. To address this need, Li et al. [7] explored the use of library/API functions, treated the source code as text, and introduced the concept of a “code gadget,” an intermediate variable that encapsulates program content. This flexible and adaptive approach effectively accommodates diverse program structures and vulnerability types, and, by employing BiLSTM, achieved significant success in detecting vulnerabilities. Furthermore, the rising popularity of attention mechanisms has advanced vulnerability detection. Liu [8]

integrated fuzzy oversampling (FOS) with deep learning to address class imbalance by using a random forest algorithm for classification and Bi-LSTM for detection. Russell [9] combined CNN and LSTM by constructing datasets from open-source code bases and applying static and dynamic analyses along with submission information and error report tags to generate labels, thereby achieving high accuracy and recall rates. Li [10] devised a lightweight detection method that extracts operational semantic features from functions to distinguish vulnerable functions from normal ones using BLSTM, achieving high classification accuracy—with F1 scores of 91% for C/C++ programs and 91.5% for Python. Traditional sequence-based methods, which focus solely on linear code relationships, often overlook complex interactions such as call jumps.

In contrast, the Full Dependency Program Graph (FDPG) introduced in this paper not only captures linear sequences but also effectively considers nonlinear relationships, such as call jumps, thereby significantly enhancing the model's understanding of code structure and behavior.

2.2 Graph-based methods

Graph structures naturally express multi-level and multi-type information, such as control and data dependencies, simultaneously retaining rich contextual details of the program. Duan [11] employed an attention mechanism combined with a fine-grained approach to generate a code property graph, encode it into a feature matrix, and then utilize a graph attention network for detection, achieving higher accuracy and lower false positive rates. Hin [12] developed a novel graph neural network-based method for statement-level vulnerability detection that effectively leverages both control and data dependencies to enhance performance. This method strikes a better balance between function-level and statement-level information, thereby improving prediction accuracy; extensive evaluations on real-world C/C++ vulnerability datasets further demonstrated its effectiveness. Cheng [13] leveraged deep graph neural networks for software vulnerability detection by extracting and preserving both structured and unstructured information from customized program fragments—referred to as XFGs—which include critical control and data dependencies for identifying vulnerabilities at the program level. Yan [14] introduced a hierarchical attention network (HAN-BSVD) for binary software vulnerability detection that preserves features through an instruction embedding network composed of Bi-GRU and word attention modules, while a feature extraction network consisting of Text-CNN and spatial attention modules captures local features and highlights key areas; evaluations on the Juliet Test Suite and ICLR19 datasets yielded superior results

compared to alternative methods. Steenhoek [15] designed a graph learning framework inspired by data flow analysis and embedding techniques to simulate data flow computation, thereby maintaining high F1 scores, precision, and recall even when trained with limited data. To further reduce false positives and false negatives, some researchers have combined the advantages of static and dynamic analyses to enhance the robustness of vulnerability detection. Zhou [16] merged slice-level granularity with the comprehensive semantics of function-level analysis to minimize both false positives and false negatives by combining weights and probabilities to score each slice and enhance its vulnerability representation. Assis [17] applied code metrics to quantify features such as size, coupling, and complexity, thereby effectively detecting vulnerabilities related to pointer usage (PU). Wu [18] drew inspiration from computer vision by treating source code graphs as social networks and employing image analysis alongside three centrality measures to train a convolutional neural network, yielding results superior to those of eight other vulnerability detectors. To address differences in data distribution across projects and improve overall detection performance, many methods have incorporated strategies such as cross-domain learning, pre-training, and hybrid networks in their model designs. Lin [19] adopted a cross-dataset approach in which two deep learning networks were trained on different data sources; labeled data were subsequently used to obtain feature representations from these networks, which were combined to train a random forest classifier. The target project's test set was then input into each network to generate feature representations for the trained classifier, achieving reliable classification results. Zhang [20] proposed a cross-domain vulnerability detection method that learns from a project rich in vulnerability labels to rapidly predict vulnerabilities in another project by reducing data distribution discrepancies through domain adaptation representation learning and employing a graph attention network for training, thereby enhancing cross-domain performance. Cheng [21] developed an embedding method that selectively retains feasible program dependency paths via a pre-trained path embedding model based on self-supervised contrastive learning; it was evaluated on 288 actual projects, achieving a high recall rate. Li [22] devised a hybrid neural network-based framework for automatic source code vulnerability detection by utilizing a lower-level virtual machine intermediate representation and backward program slicing to transform the input into an intermediate representation with explicit structural information, then applying convolutional and recurrent neural networks to detect vulnerabilities on the SARD dataset—surpassing other advanced methods. Lipp [2] proposed an automated and repeatable approach to address the issue of incorrect code granularity in

approximate vulnerability detection by demonstrating that the function level is the most suitable granularity for evaluating static code analyzers on CVE-based benchmark datasets.

In our proposed graph-based vulnerability detection method, the MLMA module employs a message passing mechanism that effectively aggregates multi-order node information and reduces information loss. This approach not only preserves the contextual details of the program but also enhances the robustness of node feature representations, thereby providing the essential rich program features for vulnerability analysis.

2.3 Large language model based approach

With the continuous evolution of large language models (LLMs), their computational and analytical capabilities have significantly strengthened, rendering them vital tools in vulnerability detection. In exploring LLM applications, Mathews et al. [23] developed a benchmark repository for detecting and classifying existing Android vulnerabilities, created a Python package, and proposed extending the system to a multi-agent framework. Sun et al. [24] enhanced LLM reasoning about vulnerabilities through techniques such as knowledge retrieval, context supplementation, prompt engineering, and instruction tracking. Mao et al. [25] integrated perspectives from both developers and testers into LLMs, thereby simulating a real code review process to facilitate thorough vulnerability classification. Steenhoek et al. [26] designed three vulnerability detection templates—bug fix patches, CVE descriptions, and static analyzers—and analyzed 287 LLM responses to construct a dataset for LLM-based detection. Zhou et al. [27] surveyed modifications in LLMs for vulnerability detection and repair, identified current challenges, and highlighted promising future research directions. Additionally, Zhou [27] proposed experiments for various tasks by incorporating task role descriptions, project information, and Common Weakness Enumeration (CWE) data. Zhang et al. [28] leveraged structural and sequential auxiliary information to enhance ChatGPT, utilizing its multi-round dialogue retention for designing vulnerability detection prompts and validating their effectiveness across two datasets. Feng et al. [29] devised an adaptive vulnerability detection scheme for large-scale smart contract models, achieving an average detection time of less than 1 ms per vulnerability. Lu et al. [30] further advanced LLM-based vulnerability detection by incorporating contextual and code graph structural information and developing a retrieval method to improve code relevance identification.

Although these methods often focus on shallow analyses of code snippets—primarily concentrating on local

vocabulary and grammatical patterns—they exhibit limitations when addressing code with complex control and data flows. In contrast, the introduction of the Full Dependency Program Graph (FDPG) and the design of the Multi-Level Message Aggregation (MLMA) module in our approach effectively address these shortcomings by accommodating and managing complex flows.

3 Framework

The core architecture of GTVD is illustrated in Fig. 1. The model comprises three key components: (a) the Full Dependency Program Graph (FDPG) generation and vectorization module; (b) the feature extraction module, which integrates both Multi-Level Message Aggregation (MLMA) and Weighted Aggregation (WAG); and (c) the vulnerability detection module. These components are designed to effectively capture the pattern characteristics of vulnerability data. Initially, the source code is transformed into an FDPG and vectorized. Node messages are then aggregated through the MLMA module, and the graph features are subsequently refined via weighted processing in the WAG module. These features are fed into a multi-layer perceptron (MLP), which maps the graph features to the output space for regression analysis and vulnerability classification. Below are the definitions and detailed descriptions of each module presented in this article:

Definition 1: Full Dependency Program Graph (FDPG): Let $G = (V, E)$, where

$$V = \{V_i | V_i \in V_{AST}\} \text{ and}$$

$$E = \{E_i | E_i \in E_{AST} \cup E_{CFG} \cup E_{PDG} \cup E_{AST^{-1}} \cup E_{CFG^{-1}} \cup E_{PDG^{-1}} \cup E_{ii}\}$$

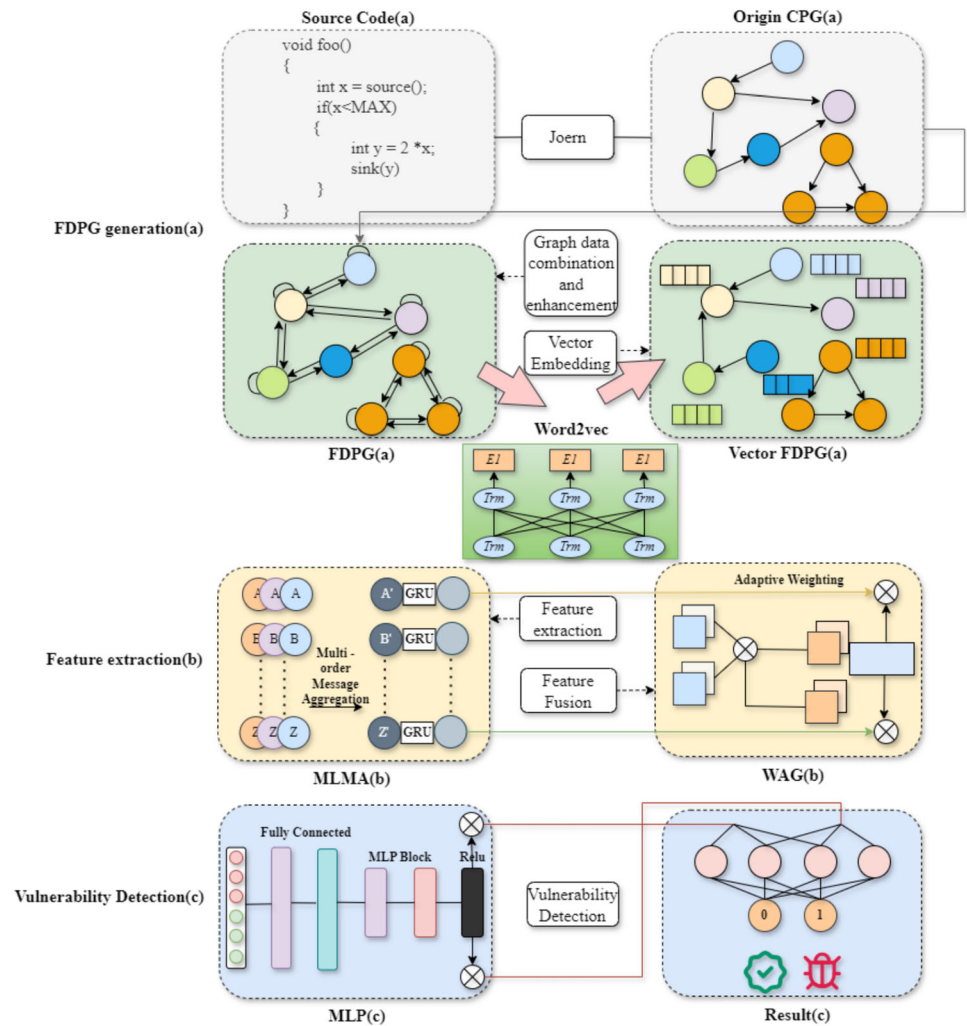
. This graph encompasses all dependencies (control and data), reverse edges, and self-loops, thereby capturing the structural nuances of the program.

Definition 2: Self-Loop Edge: A self-loop edge is one in which the starting and ending nodes are identical; that is, each node v_i points to itself ($V_i \rightarrow V_i$).

Definition 3: Reverse Edge: For a given forward edge ($V_i \rightarrow V_j$), its reverse edge represents the opposite direction ($V_j \rightarrow V_i$).

3.1 Fully dependent program graph generation

Before transforming the source code into a graph-based intermediate representation, we performed several preprocessing steps to enhance data accuracy and consistency, thereby reducing the impact of extraneous factors on model training. Initially, we removed all comments from the code because they often contain information unrelated to the

Fig. 1 The framework of the GTVD

code's logic. Subsequently, we standardized variable and function names by uniformly replacing the original identifiers with structured names, such as converting variable names to "variable_1," "variable_2," and standardizing function names to "method." This processing not only eliminates deviations caused by differences in individual coding styles but also prevents the model from mechanically memorizing specific code content, thereby enabling it to focus on learning the structure and logical relationships within the code. Figure 2 illustrates the processed data.

This study utilizes the open-source tool Joern to analyze and process source code, thereby generating a Code Property Graph (CPG). The CPG integrates multiple graphs—namely the Abstract Syntax Tree (AST), Control Flow Graph (CFG), and Program Dependency Graph (PDG)—each of which represents the code from a different perspective. These diverse representations help reveal key structural and semantic information that might be overlooked if only a single graph format were employed. In contrast, the full dependency program graph (FDPG)

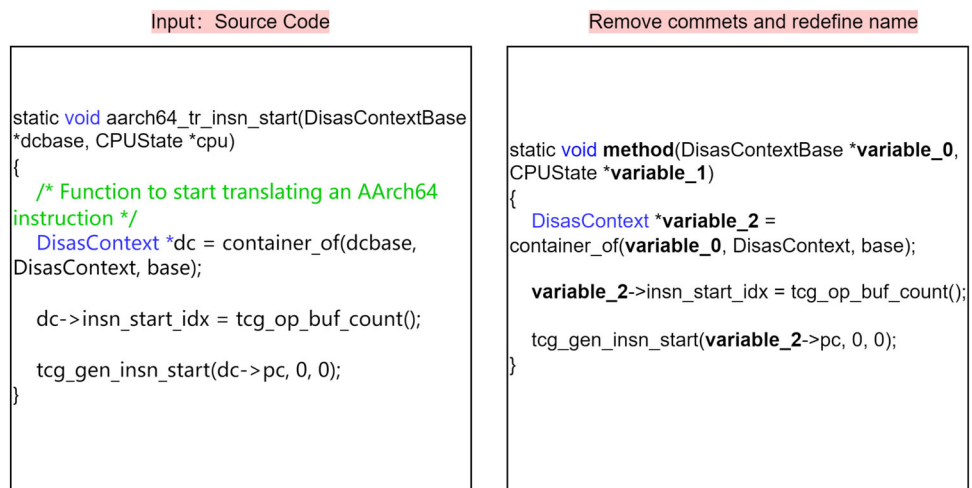
proposed in this paper integrates multiple dimensions of code representation, providing a comprehensive perspective that captures both the structure and semantic relationships within the code by introducing self-loop edges and reverse edges to enhance the expressiveness of the graph data.

The addition of self-loop edges ensures that nodes can effectively integrate their own features at each layer of the neural network, preventing their information from being masked by that of neighboring nodes during aggregation. Simultaneously, a reverse edge ($V_j \rightarrow V_i$) is added to each forward edge ($V_i \rightarrow V_j$). The presence of the reverse edge allows for the simultaneous consideration of both forward and reverse flows during the information transmission process, enabling a more comprehensive understanding of the code's logical relationships. Let

$$E_{self} = \{(V_i, V_i) | V_i \in V_{AST}\}$$

$$E_{reverse} = \{(V_j, V_i) | V_i, V_j \in V_{AST}\}$$

Fig. 2 Data preprocessing



here E_{self} denotes the set of self-loop edges, and $E_{reverse}$ denotes the set of reverse edges. These definitions indicate that a self-loop edge is added to every node, and for each forward edge in the graph (the edge of the code property graph), a corresponding reverse edge is introduced. See Fig. 3 for a specific example. The generation algorithm for the FDPG is presented in Algorithm 1.

In Fig. 3, the blue edge represents AST, the red edge represents CFG, the orange edge represents PDG, and the green edge represents FDPG.

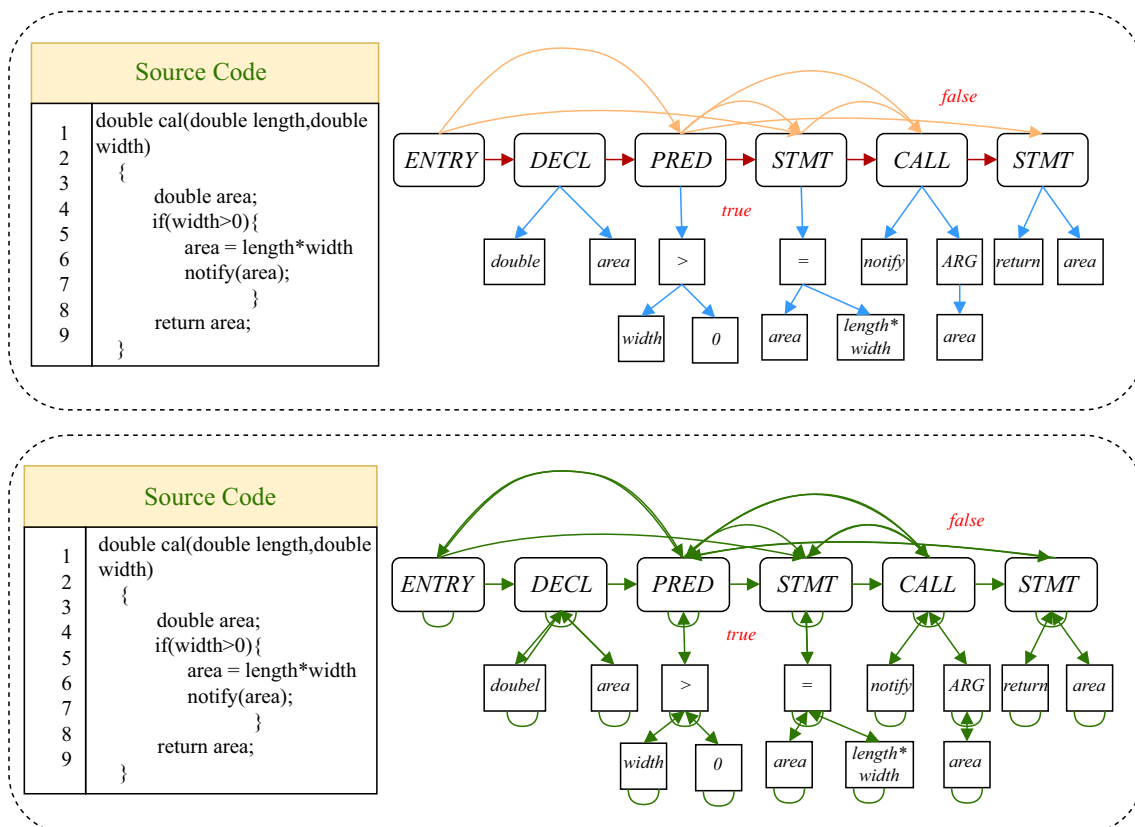


Fig. 3 Sample Code of CPG and FDPG

Algorithm 1 FDPG Generation Algorithm

Input: CPG adjacency lists, Number of nodes, Self-loop edge type

Output: FDPG

Procedure GenerateFDPG(CPG_AdjLists, NumNodes, SelfLoopType)

01. Initialize an empty graph FDPG
02. for each node in NumNodes do
03. Add node to FDPG
04. if SelfLoopType is enabled then
05. Add self-loop to node in FDPG
06. end if
07. end for
08. for each adjacency list in CPG adjLists do
09. for each edge in adjacency list do
10. Add edge to FDPG
11. end for
12. end for
13. for node in FDPG do
14. for neighbor in FDPG.Adj[node] do
15. if not exists a reverse edge then
16. Add reverse edge between node and neighbor
17. end if
18. end for
19. end for
20. return FDPG

3.2 Multi-level message aggregation

After normalizing identifiers and removing noise to generate graph data, we employ the Word2Vec [31] model to integrate each node's categorical features with its attribute information, subsequently vectorizing these combined features to obtain node embeddings. These embeddings are then processed through our multi-order message aggregation module, wherein varying aggregation orders create distinct receptive fields—higher-order aggregations capture broader contextual patterns crucial for comprehensive feature extraction. The adaptive weighting mechanism dynamically assigns importance scores to nodes based on their learned representations, ultimately generating a unified graph-level embedding. The complete aggregation workflow is depicted in Fig. 4, and its algorithmic implementation is presented in Algorithm 2.

Algorithm 2 Multi Level Message Aggregation and node embedding update

Input: $G(V,E)$, NodeEmbeddings, MessagePerType, EdgeTypeToMessageTargets, D (Depth of message passing)

Output: NewNodeEmbeddings

Procedure:

01. Initialize Messages and Targets:
02. $M \leftarrow \text{concatenate}(\text{MessagesPerType})$
03. $T \leftarrow \text{concatenate}(\text{EdgeTypeToMessageTargets})$
04. Message Aggregation: For each layer δ from 0 to $D-1$
05. $\text{AggregatedMessages} \leftarrow M$ (if $\delta=0$)
06. $\text{AggregatedMessages} \leftarrow \text{AGG}(\text{AggregatedMessages}, T, |V|)$
07. If $\delta < D-1$:
08. $\text{AggregatedMessages} \leftarrow \text{gather}(\text{AggregatedMessages}, T)$
09. Update Node Embeddings:
10. $\text{NewNodeEmbeddings}_\delta \leftarrow \text{GRUCell}(\text{AggregatedMessages}, \text{NodeEmbeddings})$

Functions:

01. $\text{AGG}(\cdot)$: An aggregation function that combines information from different nodes.
02. $\text{gather}(\cdot)$: Rearranges messages according to target indices for the next aggregation layer.
03. $\text{GRUCell}(\cdot)$: Gated Recurrent Unit for updating node embeddings

3.3 Weighted aggregation vulnerability detection

This module implements node feature weighting via a multi-head attention mechanism combined with a two-layer multilayer perceptron (MLP) with hidden dimensions of 64 and 32. This architecture dynamically adjusts node contribution weights during information aggregation, thereby significantly enhancing the network's ability to accurately model and capture data characteristics. The formulation is presented in Eq. (1).

$$W_h = \sigma(X * W_h^{(s)}) \text{ for } h = 1, 2, 3, 4 \quad (1)$$

In this formulation, $W_h^{(s)}$ denotes the weight matrix corresponding to the h attention head, which extracts relevant information from the feature matrix X . The activation function σ computes the significance weights for each node, and W_h subsequently represents the quantified node importance after weight computation. The node feature transformation is detailed in Eq. (2).

$$H_i = \Phi(X_i * W_t + b_t) \quad (2)$$

In this formulation, X_i denotes the feature vector of node i . The parameters W_t and b_t correspond to the weight

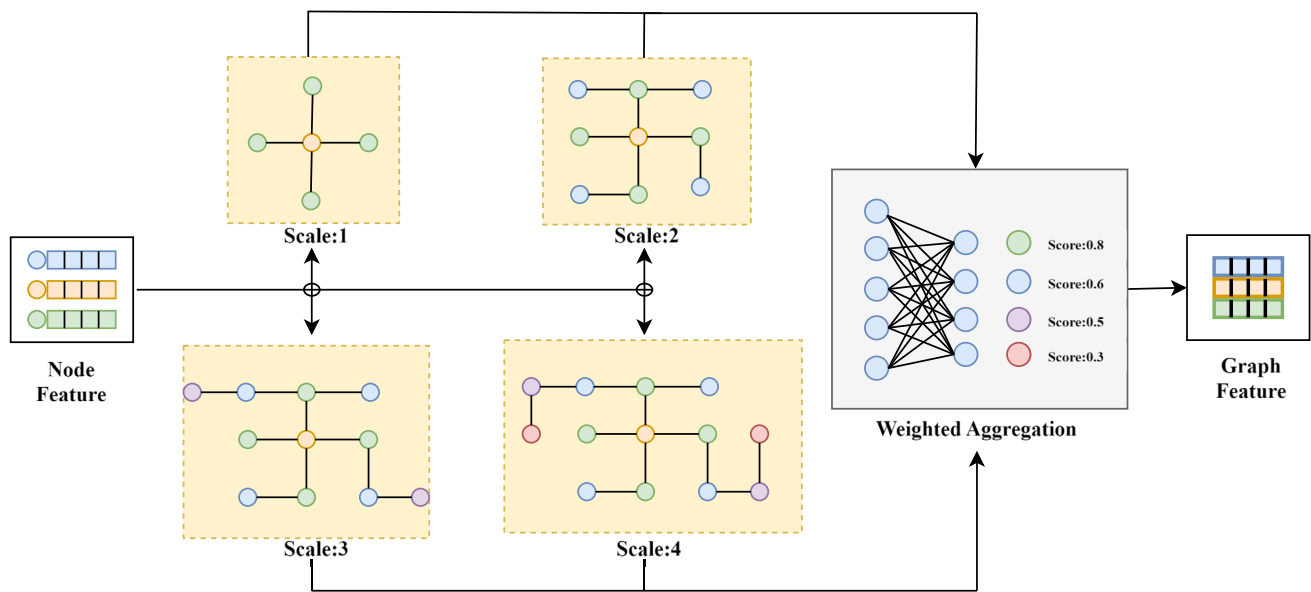


Fig. 4 The framework of MLMA

matrix and bias vector of the transformation MLP, respectively. The activation function Φ introduces nonlinearity following the feature transformation, producing the updated node representation H_i .

The weighted node features are then aggregated to generate graph-level representations. This adaptive weighting mechanism ensures that each node's contribution reflects its structural significance within the graph. Additionally, value constraints are applied to node representation transformations to prevent activation explosion or excessive attenuation. The network employs the Gaussian Error Linear Unit (GELU) activation to enhance nonlinear processing capabilities, combining the advantages of Sigmoid's smoothness and ReLU's gradient preservation while mitigating vanishing gradient issues, as shown in Eq. (3).

$$GELU(X) = 0.5 \times x \left(1 + \tanh \left[\sqrt{\frac{2}{\pi}} (x + 0.044715x^3) \right] \right) \quad (3)$$

To prevent overfitting, dropout layers with a probability of 0.5 are inserted after each hidden layer. Finally, the graph embeddings are processed by an MLP with Softmax activation for binary classification, where the output probabilities are normalized to the range [0, 1] for vulnerability prediction.

4 Experiments

4.1 Dataset

We evaluate the GTVD model using three real-world vulnerability datasets: FFmpeg&Qemu [32], BigVul [33], and Reveal [34]. While the BigVul and Reveal datasets exhibit significant class imbalance, the FFmpeg&Qemu dataset maintains a relatively balanced class structure, with a similar number of positive and negative samples. These complementary datasets enable a comprehensive evaluation across different data distributions. Our experimental framework consists of four components: baseline model comparison, convergence analysis, multi-stage aggregation verification, and an ablation study. Detailed dataset statistics are provided in Table 1, and additional vulnerability data specifications are available via the referenced repository links.

Table 1 Dataset statistics

Dataset	Sample	Vul	Non-Vul	Vul ratio (%)
FFmpeg&Qemu	22,361	10,067	12,294	45.02
BigVul	179,299	10,547	168,752	5.88
Reveal	18,169	1664	16,505	9.16

https://github.com/ZeoVan/MSR_20_Code_vulnerability_CSV_Dataset

<https://drive.google.com/file/d/1x6hoF7G-tSYxg8A>

[FybggypLZgMGDNHf/editChromium_And_Debian_Vulnerability_Data](https://drive.google.com/file/d/1x6hoF7G-tSYxg8A)

4.2 Training details

The experimental parameters are detailed in Table 2. The dataset was partitioned into training, validation, and test sets in an 8:1:1 ratio. The experiments were conducted using Python 3.8 and PyCharm 2020.3.5. The operating environments include a 64-bit Windows 10 system and a 64-bit Ubuntu 20.04 system, both running on an Intel® Core™ i5-12450 CPU at 2.50 GHz. The learning rate was set to 0.001. Figure 5 illustrates the training and validation error curves for both the FFmpeg&Qemu and BigVul datasets. The graphs demonstrate a decrease in error with an increasing number of training cycles, achieving convergence around Epoch 90.

The horizontal axis in the figure represents the training rounds, while the vertical axis denotes the loss values. As illustrated, with an increasing number of training rounds, the loss values for the models trained on the three datasets gradually decrease and converge around Epoch 90. This observation demonstrates the feasibility of our method and confirms the model's effectiveness.

4.3 Evaluation metrics

The evaluation metrics used to assess GTVD's vulnerability detection performance include Accuracy, Precision, Recall, and F1-score, as defined in Eqs. (4) to (7). Detailed hyperparameters are presented in Table 2. Higher values of these metrics indicate better vulnerability detection capabilities.

TP (True Positives): The number of samples accurately predicted as positive.

TN (True Negatives): The number of samples accurately predicted as negative.

FP (False Positives): The number of samples incorrectly predicted as positive.

FN (False Negatives): The number of samples incorrectly predicted as negative.

Table 2 Parameters of our proposed model

Parameters	Value
Number of hidden units	256
Batch_size	32
Epoch	100
Learing_rate	0.001
Optimizer	Adma
Activation funtion	gelu
Dropout	0.2

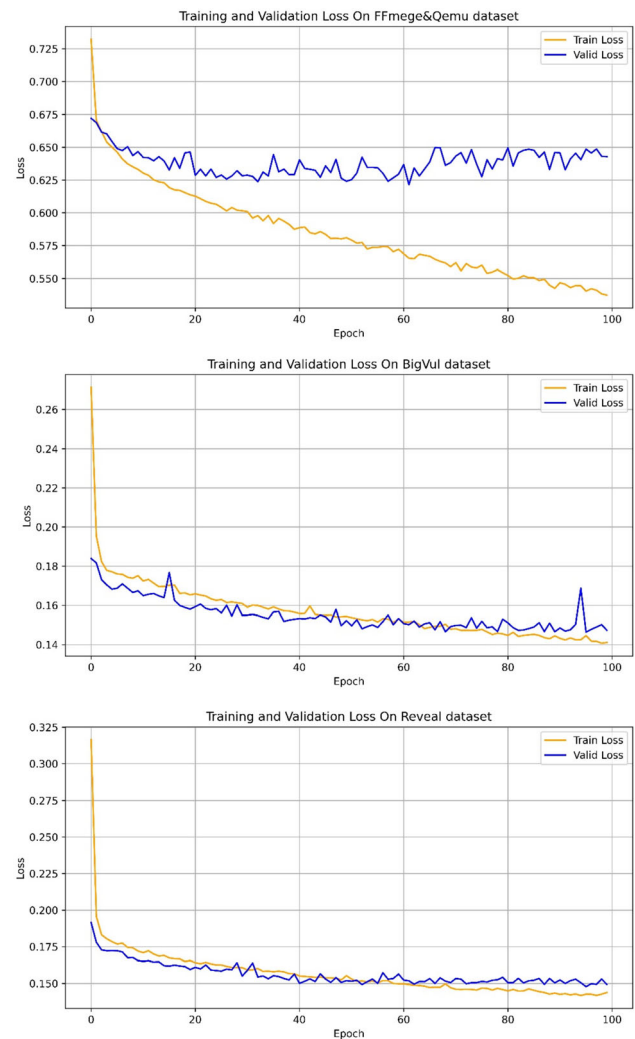


Fig. 5 Loss of different datasets on train and valid

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN} \quad (4)$$

$$Precision = \frac{TP}{FP + TP} \quad (5)$$

$$Recall = \frac{TP}{FN + TP} \quad (6)$$

$$F1 - score = \frac{2 * Precision * Recall}{Precision + Recall} \quad (7)$$

4.4 Baseline

To establish the superiority of our model, we compared it against the following baseline methods:

VulDeePecker [7]: Employs code gadgets as an intermediate representation to depict source data and trains the model using a bidirectional LSTM (Long Short-Term Memory) neural network for vulnerability detection.

SySeVR [35]: An advanced successor to VulDeePecker, this method transforms source code into a graph that captures both syntactic and semantic information for its intermediate representation, employing a bidirectional gated recurrent unit (BGRU) to learn features and classify vulnerabilities.

Russell [9]: Integrates the neural feature representation of lexical function source code with an ensemble classifier (the random forest) and employs a convolutional neural network to detect function vulnerabilities.

IVDetect [36]: Represents code using a program dependency graph (PDG) and leverages a graph convolutional network with feature attention for fine-grained vulnerability detection, thereby enhancing the interpretability of vulnerability statements.

BovdGFE [37]: Proposes a novel representation structure called Code Representation Sequence (CORS), which combines the control flow, data dependencies, and grammatical structure of fragile code to reduce information loss during code representation.

AMPLE [38]: Reduces the distance between nodes through graph simplification and employs an edge-aware graph convolutional network module to fuse heterogeneous edge information into the node representation, thereby better capturing the relationships between distant nodes.

GRACE [30]: Using the big oracle model as the basis, the vulnerability detection capability of the big oracle model is enhanced by combining the graph structure information and contextual information in the code, while taking into account the semantic, lexical and syntactic similarities of the target code.

4.5 Result analysis

We conducted comparative experiments using GTVD and baseline models on three different datasets:

Table 3 The comparative results on FFmpeg&Qemu datasets

Metrics (%)	Accuracy	Precision	Recall	F1-Score
VulDeePecker	49.91	46.05	32.55	38.14
SySeVR	47.85	46.06	58.81	51.66
Russell	57.60	54.76	40.72	46.71
IVDetect	57.26	52.37	57.55	54.84
BovdGFE	56.46	50.18	50.63	47.27
AMPLE	62.16	55.64	83.99	66.94
GRACE	59.78	53.94	82.13	65.11
GTVD	61.39	56.27	60.53	52.32

Bold representing the item with the highest value in the current column

Table 4 The comparative results on BigVul datasets

Metrics (%)	Accuracy	Precision	Recall	F1-Score
VulDeePecker	81.19	38.44	12.75	19.15
SySeVR	90.10	30.91	14.08	19.34
Russell	86.85	14.86	26.97	19.17
IVDetect	87.14	17.22	34.04	22.87
BovdGFE	88.94	25.38	28.17	30.29
AMPLE	93.14	22.98	34.58	32.11
GRACE	90.73	32.52	39.08	35.50
GTVD	93.91	38.02	56.88	32.27

Bold representing the item with the highest value in the current column

Table 5 The comparative results on Reveal datasets

Metrics (%)	Accuracy	Precision	Recall	F1-Score
VulDeePecker	76.37	21.13	13.10	16.17
SySeVR	68.51	16.21	52.68	24.79
Russell	81.77	31.55	61.14	41.62
BovdGFE	88.72	40.16	43.73	35.42
AMPLE	92.71	51.06	46.15	48.48
GRACE	89.73	33.21	61.53	43.13
GTVD	90.52	61.76	68.38	48.83

Bold representing the item with the highest value in the current column

FFmpeg&Qemu, BigVul, and Reveal. Detailed experimental results are presented in Tables 3, 4 and 5.

Experimental results show that GTVD performs well in C/C++ source code vulnerability detection. Although not all evaluation indicators are better than other methods, the comprehensive evaluation on different datasets is the best, and the recall rate (Recall) and F1-Score on the FFmpeg&Qemu dataset are slightly lower than the latest baselines AMPLE and GRACE. This difference may be attributed to the fact that AMPLE uses a graph simplification module, which shortens the distance between nodes and enhances robustness; while GRACE combines LLM, and the complex structure of LLM makes the vulnerability detection effect better. GTVD has excellent comprehensive representation. Especially on the BigVul dataset, the recall rate of GTVD is improved by 17.8% compared with the latest GRACE, which strongly proves the superiority of GTVD.

To verify the effectiveness of the proposed multi-level message aggregation module, we designed a comparative experiment to examine the impact of different message aggregation scales on GTVD's vulnerability detection performance. By adjusting the aggregation order, we

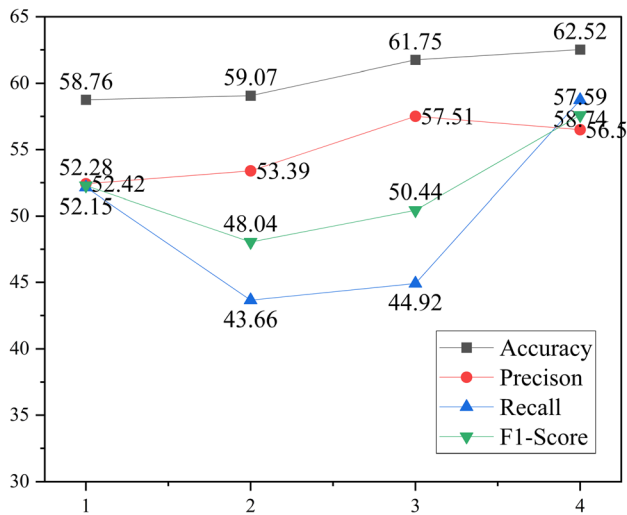


Fig. 6 Comparison of message aggregation scales in FFmpeg&Qemu dataset

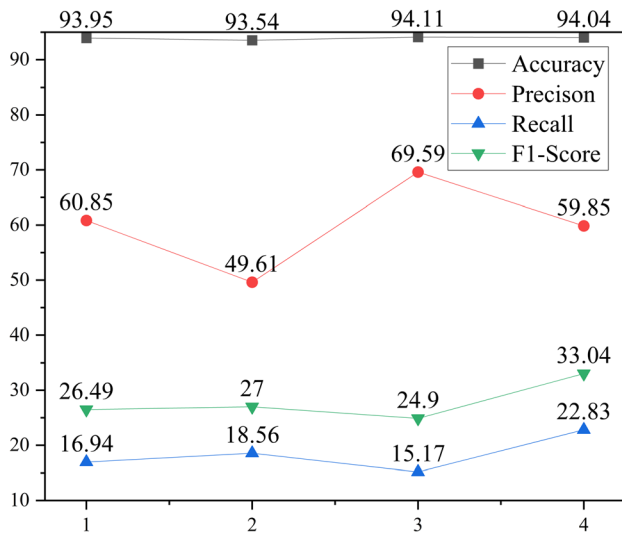


Fig. 7 Comparison of message aggregation scales in BigVul dataset

evaluated the model's performance under varying levels of graph structure information. The experimental results are presented in Figs. 6, 7 and 8.

From the data presented, it is evident that increasing the aggregation scale improves the evaluation metrics across the two datasets. At the fourth order, the overall performance of various indicators reaches its optimum. This improvement occurs because different aggregation orders provide nodes with varying receptive fields during message aggregation. Higher orders enable nodes to incorporate richer features from neighboring nodes, thereby enhancing the accuracy of vulnerability detection. On three different datasets, the F1-Score at the fourth order is 6.32%, 6.55%, and 5.71% higher than that at the first order, respectively, demonstrating that a higher message aggregation order

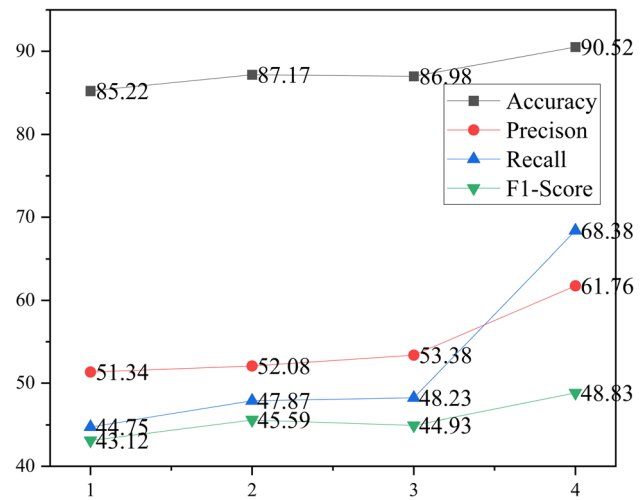


Fig. 8 Comparison of message aggregation scales in Reveal dataset

enables the model to achieve better vulnerability detection performance.

4.6 Ablation experiment

To clarify the specific contributions of the Full Dependency Program Graph (FDPG), Multi-Scale Message Aggregation (MLMA) module, and Weighted Aggregation (WAG) to the model's performance, we conducted a series of comprehensive ablation studies. These studies evaluated the performance of GTVD in configurations without FDPG versus with FDPG, assessed the impact of including and excluding WAG, and analyzed the performance changes with and without Multi-Scale Message Aggregation. The introduction of each module is as follows:

FDPG: Provides a more comprehensive representation of source code information by incorporating additional graph data features.

MLMA: Allows each node in the message passing process to acquire a richer receptive field, thereby integrating more comprehensive neighbor node features for vulnerability detection and improving detection accuracy.

Table 6 Multi-module ablation experiment on FFmpeg + Qemu dataset

Metrics (%)	Accuracy	Precision	Recall	F1-Score
w/o FDPG	58.04	52.41	54.39	53.38
w/o WAG	59.72	55.46	56.07	55.49
w/o MLMA	60.30	54.78	56.12	55.43
GTVD	62.52	56.50	58.74	57.59

Bold representing the item with the highest value in the current column

Table 7 Multi-module ablation experiment on BigVul dataset

Metrics (%)	Accuracy	Precision	Recall	F1-Score
w/o FDPG	87.71	53.09	18.26	27.32
w/o WAG	89.17	55.52	20.33	27.12
w/o MLMA	90.89	56.42	20.41	30.07
GTVD	94.04	59.85	22.83	33.04

Bold representing the item with the highest value in the current column

Table 8 Multi-module ablation experiment on Reveal dataset

Metrics (%)	Accuracy	Precision	Recall	F1-Score
w/o FDPG	81.61	49.17	41.82	39.18
w/o WAG	85.96	50.49	43.91	42.6
w/o MLMA	88.57	53.83	45.24	46.55
GTVD	90.52	61.76	68.38	48.83

Bold representing the item with the highest value in the current column

WAG: Enables the network to dynamically adjust the weight of each node based on its contribution to the entire graph, significantly enhancing the model's ability to capture and express complex graph structures.

The experimental results are shown in Tables 6, 7 and 8.

From the data in the table, it is evident that omitting the FDPG, MLMA and WAG, modules leads to varying degrees of degradation in the model's vulnerability detection performance. This finding underscores the effectiveness of the multi-module approach proposed for vulnerability detection. Notably, FDPG exhibits the greatest impact on model performance, with an 26.56% improvement in accuracy on the Reveal dataset. This result demonstrates that the proposed FDPG provides a superior representation of code data.

5 Conclusion

In this paper, we propose a model called GTVD for source code vulnerability detection, which employs graph data to represent the relationships between nodes in C/C++ source code as an intermediate representation for detecting potential security vulnerabilities. By combining various attribute graphs and incorporating vectorized representations of reverse edges and self-loop edges, we construct a Full Dependency Program Graph (FDPG) that can more comprehensively capture function calls and dependencies in the code. Additionally, we utilize a Multi-Level Message Aggregation (MLMA) module to adjust the order of node

feature extraction, allowing nodes to aggregate information from a broader range of neighbors and obtain richer features. Through Weighted Aggregation (WAG), nodes are dynamically weighted based on their importance within the graph, generating a final graph representation. This representation is then fed into a multilayer perceptron (MLP) for classification, significantly improving the accuracy and effectiveness of vulnerability detection. Experimental results demonstrate that GTVD surpasses most existing models on three large real-world datasets, achieving 1.61%, 3.18%, and 0.79% accuracy improvements on each dataset, respectively, compared to the latest baselines. In addition, GTVD's recall improves by 17.8% and 6.85% on the BigVul and Reveal datasets, respectively. In the future, we plan to extend GTVD to programming languages beyond C/C++ to enhance its generalizability and practical applicability.

Acknowledgements This work was supported by National Natural Science Foundation of China (No. 62376240), S&T Program of Hebei (Nos. 226Z0701G, 236Z0702G, 236Z0304G), the Natural Science Foundation of Hebei Province (Nos. F2022203026, F2022203089), Science Research Project of Hebei Education Department (No. BJK2022029) and Innovation Capability Improvement Plan Project of Hebei Province (No. 22567637H). The authors are grateful to the valuable comments and suggestions of the reviewers.

Author contributions Haitao He: supervision, conceptualization, methodology, writing—review & editing. Shibo Li: software, formal analysis, writing—original draft, data curation. Yuxiang Li: visualization, investigation. Yang Li: software, validation.

Data availability No datasets were generated or analysed during the current study.

Declarations

Conflict of interest The authors declare no competing interests.

References

1. Zhang, J., Pan, L., Han, Q.L., et al.: Deep learning based attack detection for cyber-physical system cybersecurity: a survey. *IEEE/CAA J. Autom. Sin.* **9**(3), 377–391 (2021)
2. Lipp, S., Banescu, S., Pretschner, A.: An empirical study on the effectiveness of static C code analyzers for vulnerability detection. In: *Proceedings of the 31st ACM SIGSOFT International Symposium on Software Testing and Analysis*, pp. 544–555 (2022)
3. Lin, G., Wen, S., Han, Q.L., et al.: Software vulnerability detection using deep neural networks: a survey. *Proc. IEEE* **108**(10), 1825–1848 (2020)
4. Zeng, P., Lin, G., Pan, L., et al.: Software vulnerability analysis and discovery using deep learning techniques: a survey. *IEEE Access* **8**, 197158–197172 (2020)
5. Wang, H., Ye, G., Tang, Z., et al.: Combining graph-based learning with automated data collection for code vulnerability detection. *IEEE Trans. Inf. Forensics Secur.* **16**, 1943–1958 (2020)

6. Bui, V.C., Do, X.C.: Detecting software vulnerabilities based on source code analysis using GCN transformer. In: 2023 RIVF International Conference on Computing and Communication Technologies (RIVF), pp. 112–117. IEEE (2023)
7. Li, Z., Zou, D., Xu, S., et al.: Vuldeepecker: a deep learning-based system for vulnerability detection (2018). arXiv preprint [arXiv:1801.01681](https://arxiv.org/abs/1801.01681)
8. Liu, S., Lin, G., Han, Q.L., et al.: DeepBalance: deep-learning and fuzzy oversampling for vulnerability detection. *IEEE Trans. Fuzzy Syst.* **28**(7), 1329–1343 (2019)
9. Russell, R., Kim, L., Hamilton, L., et al.: Automated vulnerability detection in source code using deep representation learning. In: 2018 17th IEEE International Conference on Machine Learning and Applications (ICMLA), pp. 757–762. IEEE (2018)
10. Li, R., Feng, C., Zhang, X., et al.: A lightweight assisted vulnerability discovery method using deep neural networks. *IEEE Access* **7**, 80079–80092 (2019)
11. Duan, X., Wu, J., Ji, S., et al.: VulSniper: focus your attention to shoot fine-grained vulnerabilities. In: IJCAI, pp. 4665–4671 (2019)
12. Hin, D., Kan, A., Chen, H., et al.: LineVD: statement-level vulnerability detection using graph neural networks. In: Proceedings of the 19th International Conference on Mining Software Repositories, pp. 596–607 (2022)
13. Cheng, X., Wang, H., Hua, J., et al.: Deepwukong: statically detecting software vulnerabilities using deep graph neural network. *ACM Trans. Softw. Eng. Methodol. (TOSEM)* **30**(3), 1–33 (2021)
14. Yan, H., Luo, S., Pan, L., et al.: HAN-BSVD: a hierarchical attention network for binary software vulnerability detection. *Comput. Secur.* **108**, 102286 (2021)
15. Steenhoeck, B., Gao, H., Le, W.: Dataflow analysis-inspired deep learning for efficient vulnerability detection. In: Proceedings of the 46th IEEE/ACM International Conference on Software Engineering, pp. 1–13 (2024)
16. Zou, D., Hu, Y., Li, W., et al.: mvulpreter: a multi-granularity vulnerability detection system with interpretations. *IEEE Trans. Dependable Secure Comput.* (2022). <https://doi.org/10.1109/TDSC.2022.3199769>
17. Zagane, M., Abdi, M.K., Alenezi, M.: Deep learning for software vulnerabilities detection using code metrics. *IEEE Access* **8**, 74562–74570 (2020)
18. Wu, Y., Zou, D., Dou, S., et al.: Vulcnn: an image-inspired scalable vulnerability detection system. In: Proceedings of the 44th International Conference on Software Engineering, pp. 2365–2376 (2022)
19. Lin, G., Zhang, J., Luo, W., et al.: Software vulnerability discovery via learning multi-domain knowledge bases. *IEEE Trans. Dependable Secure Comput.* **18**(5), 2469–2485 (2019)
20. Zhang, C., Liu, B., Xin, Y., et al.: CPVD: Cross project vulnerability detection based on graph attention network and domain adaptation. *IEEE Trans. Softw. Eng.* **49**(8), 4152–4168 (2023)
21. Cheng, X., Zhang, G., Wang, H., et al.: Path-sensitive code embedding via contrastive learning for software vulnerability detection. In: Proceedings of the 31st ACM SIGSOFT International Symposium on Software Testing and Analysis, pp. 519–531 (2022)
22. Li, X., Wang, L., Xin, Y., et al.: Automated software vulnerability detection based on hybrid neural network. *Appl. Sci.* **11**(7), 3201 (2021)
23. Mathews, N.S., Brus, Y., Aafer, Y., et al.: Llbzpeky: leveraging large language models for vulnerability detection (2024). arXiv preprint [arXiv:2401.01269](https://arxiv.org/abs/2401.01269)
24. Sun, Y., Wu, D., Xue, Y., et al.: LLM4Vuln: A Unified Evaluation Framework for Decoupling and Enhancing LLMs' Vulnerability Reasoning (2024). arXiv preprint [arXiv:2401.16185](https://arxiv.org/abs/2401.16185)
25. Mao, Z., Li, J., Li, M., et al.: Multi-role Consensus through LLMs Discussions for Vulnerability Detection (2024). arXiv preprint [arXiv:2403.14274](https://arxiv.org/abs/2403.14274)
26. Steenhoeck, B., Rahman, M.M., Roy, M.K., et al.: A Comprehensive Study of the Capabilities of Large Language Models for Vulnerability Detection (2024). arXiv preprint [arXiv:2403.17218](https://arxiv.org/abs/2403.17218)
27. Zhou, X., Zhang, T., Lo, D.: Large language model for vulnerability detection: emerging results and future directions. In: Proceedings of the 2024 ACM/IEEE 44th International Conference on Software Engineering: New Ideas and Emerging Results, pp. 47–51 (2024)
28. Zhang, C., Liu, H., Zeng, J., et al.: Prompt-enhanced software vulnerability detection using chatgpt. In: Proceedings of the 2024 IEEE/ACM 46th International Conference on Software Engineering: Companion Proceedings, pp. 276–277 (2024)
29. Feng, X., Liu, H., Wang, L., et al.: An Interpretable Model for Large-Scale Smart Contract Vulnerability Detection. *Blockchain: Research and Applications*, p. 100209 (2024)
30. Lu, G., Ju, X., Chen, X., et al.: GRACE: empowering LLM-based software vulnerability detection with graph structure and in-context learning. *J. Syst. Softw.* **212**, 112031 (2024)
31. Mikolov, T.: Efficient estimation of word representations in vector space, p. 3781 (2013). arXiv preprint [arXiv:1301.3781](https://arxiv.org/abs/1301.3781)
32. Zhou, Y., Liu, S., Siow, J., et al.: Devign: effective vulnerability identification by learning comprehensive program semantics via graph neural networks. In: Advances in neural information processing systems, p. 32 (2019)
33. Fan, J., Li, Y., Wang, S., et al.: AC/C++ code vulnerability dataset with code changes and CVE summaries. In: Proceedings of the 17th International Conference on Mining Software Repositories, pp. 508–512 (2020)
34. Chakraborty, S., Krishna, R., Ding, Y., et al.: Deep learning based vulnerability detection: are we there yet? *IEEE Trans. Softw. Eng.* **48**(9), 3280–3296 (2021)
35. Li, Z., Zou, D., Xu, S., et al.: Sysevr: a framework for using deep learning to detect software vulnerabilities. *IEEE Trans. Dependable Secure Comput.* **19**(4), 2244–2258 (2021)
36. Li, Y., Wang, S., Nguyen, T.N.: Vulnerability detection with fine-grained interpretations. In: Proceedings of the 29th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering, pp. 292–303 (2021)
37. Lv, X., Peng, T., Chen, J., et al.: BovdGFE: buffer overflow vulnerability detection based on graph feature extraction. *Appl. Intell.* **53**(12), 15204–15221 (2023)
38. Wen, X.C., Chen, Y., Gao, C., et al.: Vulnerability detection with graph simplification and enhanced graph representation learning. In: 2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE), pp. 2275–2286. IEEE (2023)

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Springer Nature or its licensor (e.g. a society or other partner) holds exclusive rights to this article under a publishing agreement with the author(s) or other rightsholder(s); author self-archiving of the accepted manuscript version of this article is solely governed by the terms of such publishing agreement and applicable law.



Haitao He (first author) is a professor at the School of Information Science and Engineering, Yanshan University. Her research interests include AI, Data mining, Software security analysis.



Yuxiang Li, born in 1998. He is currently a master's degree at the School of Information Science and Engineering, Yanshan University. His research interests are speech recognition and speaker verification.



Shibo Li, born in 2000. He is currently a master's degree at the School of Information Science and Engineering, Yanshan University. His research interests are software security, Data mining, and AI.



Yang Li, born in 1998. He is currently a master's degree at the School of Information Science and Engineering, Yanshan University. His current research interests include vulnerability detection and deep learning.